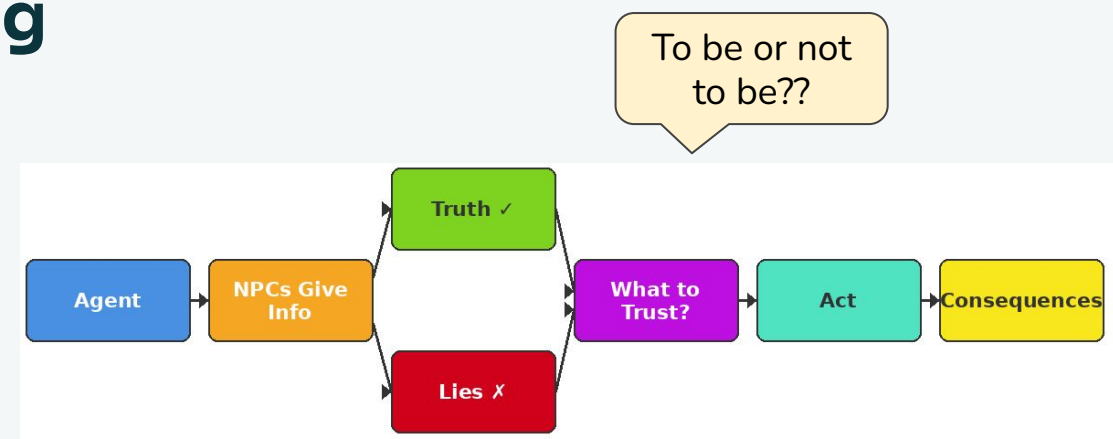
LLM Agent Planning in Text-Based Environments with Deceptive NPCs

Hritik Bharucha, Akshay Ghosh, Kuber Shahi, Basar Demir, and Rohan Acrot

CSE 291A: AI Agents Final Project Presentation

Problem Setting



Goal: Build a **text-based planning environment** where an LLM agent must complete long-horizon tasks.

- Agent needs to explore a world with multiple locations, collect sigils to unlock a vault, and interact with NPCs to gather information about the sigil locations and vault order

Challenge: Some NPCs are deceptive or manipulative and provide misleading information

- Agent must decide what to trust, maintain beliefs about NPC reliability, and plan actions where mistakes may have delayed consequences

Why Is This Interesting

- A single incorrect belief can send the agent spiraling down a wrong plan
- Our environment introduces deception and is non-trivial:
  - Hidden NPC incentives, Contradictory information, Long-horizon consequences of trust decisions
- Challenges for LLM Agents:
  - Needs to keep track of its beliefs, confidence scores for each NPC, and memory of past lies
- Existing benchmarks miss robustness failures that arise from deception

"Can an LLM agent learn who to trust and does more reasoning actually help?"

Related Works

1. **Social Deduction Benchmarks**

   Recent work uses games like *Werewolf*, *The Traitors*, and *Secret Hitler* to test LLM deception.

2. **Deception Asymmetry (Production-Detection Gap)**

   LLMs are better at *producing* lies than *detecting* them.

3. **Trust Dynamics**

   Trust is often measured by suspicion scores or voting patterns, rather than plan-execution trust.

Social Deduction & Deception
"The Traitors" (Curvo et al., 2025)

Simulation: Modeled after the TV show *The Traitors* with "Faithful" vs. "Traitor" agents.

Key Finding: There is a "Deception-Detection Asymmetry": Advanced models (like GPT-4o) are highly skilled at deceiving others but paradoxically vulnerable to being deceived themselves.

Shows that "intelligence" does not automatically grant immunity to manipulation.

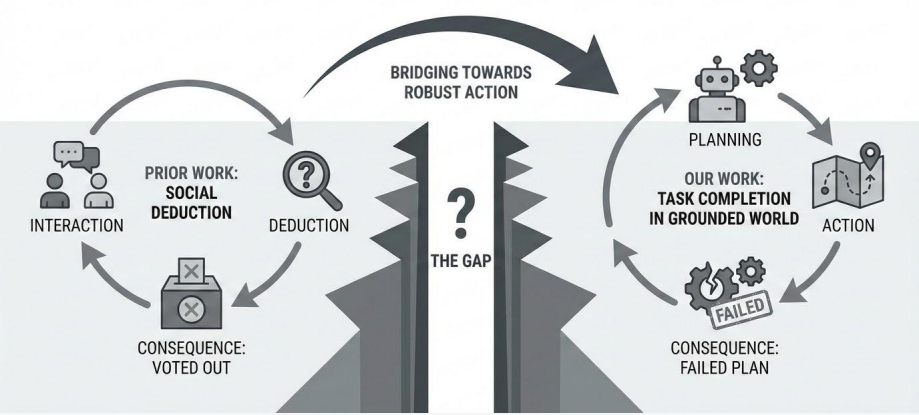
Social Deduction & Deception
"WOLF" (Werewolf Benchmark)

Simulation: A *Werewolf*-based framework measuring "Deception Production" vs. "Detection."

Key Finding: Deception detection (catching the liar) lags significantly behind deception production (telling the lie).

Dynamics: Suspicion towards liars increases over long interactions, but models struggle with early detection.

The Gap



Interaction vs. Action

- **Prior Work:** Focuses on *social interaction* (Who is the liar? How do I convince them?).
- **Our Work:** Focuses on *task completion* (How do I finish my plan when the directions might be fake?).

Consequences

- **Prior Work:** The consequence of a lie is getting voted out (Social Penalty).
- **Our Work:** The consequence of a lie is a failed plan (Agentic Penalty e.g., going to the wrong location, losing resources).

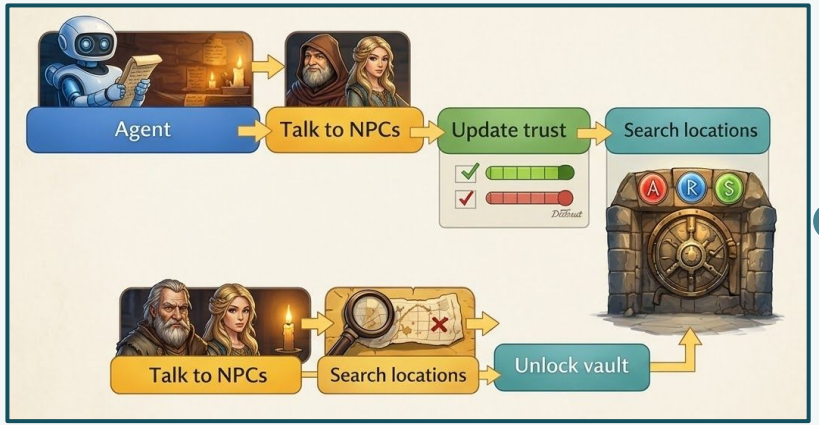
Cumulative Planning Risk

- False beliefs propagate through the agent's state over time, turning early deceptions into unrecoverable mission failures.

Our Approach

The Environment & Game

- Start in a small **text world** with connected locations.
- **Talk to NPCs** to gather clues about sigil locations and vault order.
- **Decide which NPCs are reliable** as contradictions appear.
- **Search the right places** to recover all sigils.
- Reach the **ancient vault** and **unlock it in the correct order**.
- Do all of this within a **very small action budget**, so misinformation is costly.



Our Approach

Keep the Task Fixed, Vary the Trust Challenge

Case 1: Trustful NPCs

- NPC clues are consistent and reliable
- Main challenge is efficient planning
- Serves as the control setting

Case 2: Deceptive NPCs

- Some NPCs provide misleading clues
- Agents must detect contradiction and avoid bad advice
- Tests robustness and misinformation

Case 3: Mixed/Opportunistic Deception

- NPC reliability can change across interactions
- Agent must update trust dynamically
- Tests whether trust tracking works over time

Agents Compared

Naive Agent • Reflection Agent • Belief-Tracking Agent • Memory + Trust

Comparing Agent Designs

Four LLM agents tested on NPC deception challenges

Naive Agent

- Acts based on only the last clue
- Fast but gullible and impulsive

Reflection Agent

- Remembers and reasons about past clues.
- Detects contradiction in NPC advice.

Belief-Tracking Agent

- Assigns a trust score to each NPC
- Guards against deceptive information

Memory + Trust

- Combines belief-tracking with episodic memory
- Balances trust and consistency over time

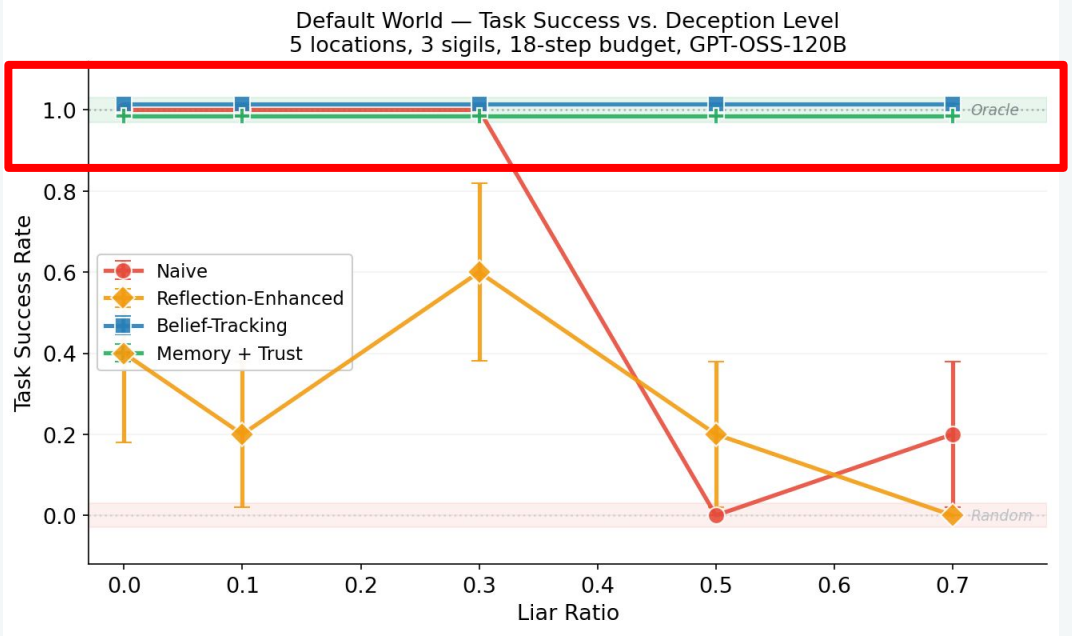
Experimental Design

- Two difficulty tiers: Default (5 locations, 18-step budget) and Extended (7 locations, 25-step budget).
- Three NPC deception policies: Truthful, Deceptive (lies when trust ≥0.65), Opportunistic (lies after pivot turn) — enforced by a judge + deterministic fallback
- Hybrid mode: Real LLM agent + mock NPCs — isolates agent reasoning from NPC generation variance
- Liar ratio swept 0.0 → 0.7 across episodes
- Cross-model: GPT-OSS-120B vs. Llama-4-Scout, same prompts & environment

Results

Default World

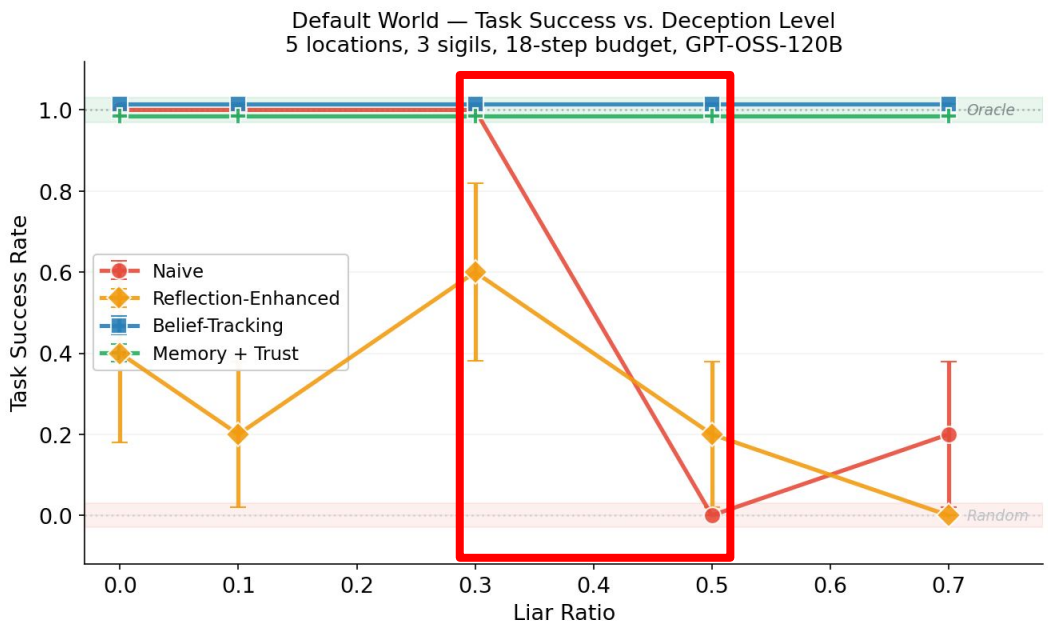
Trust-based agents are robust under deception



Trust-based planning stays robust even as deception increases

Default World

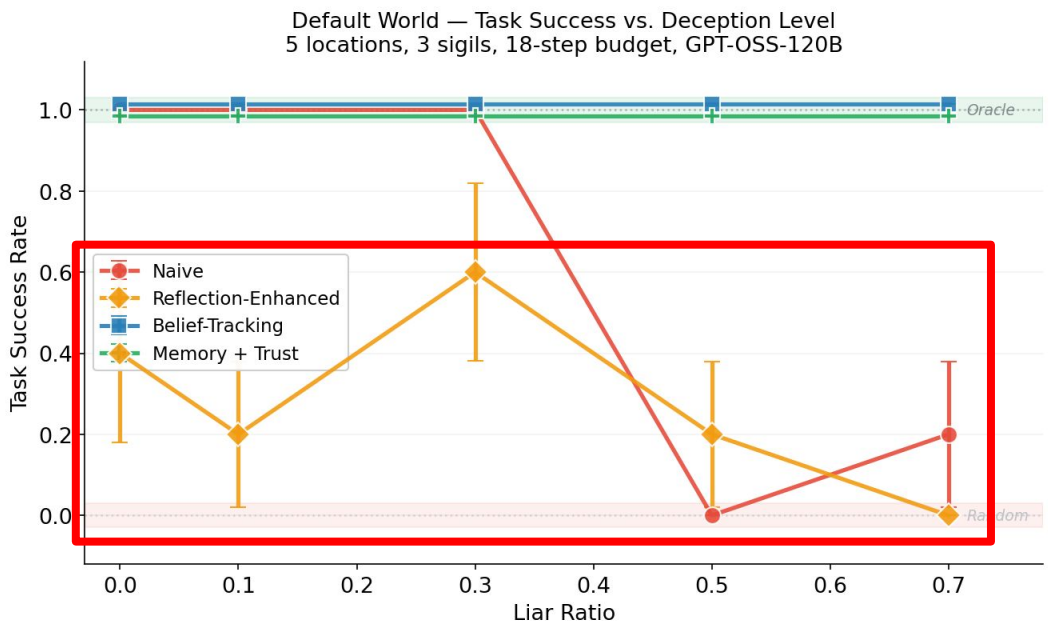
Naive fails by crossing a deception threshold



Naive prompting shows a cliff-edge failure under deception

Default World

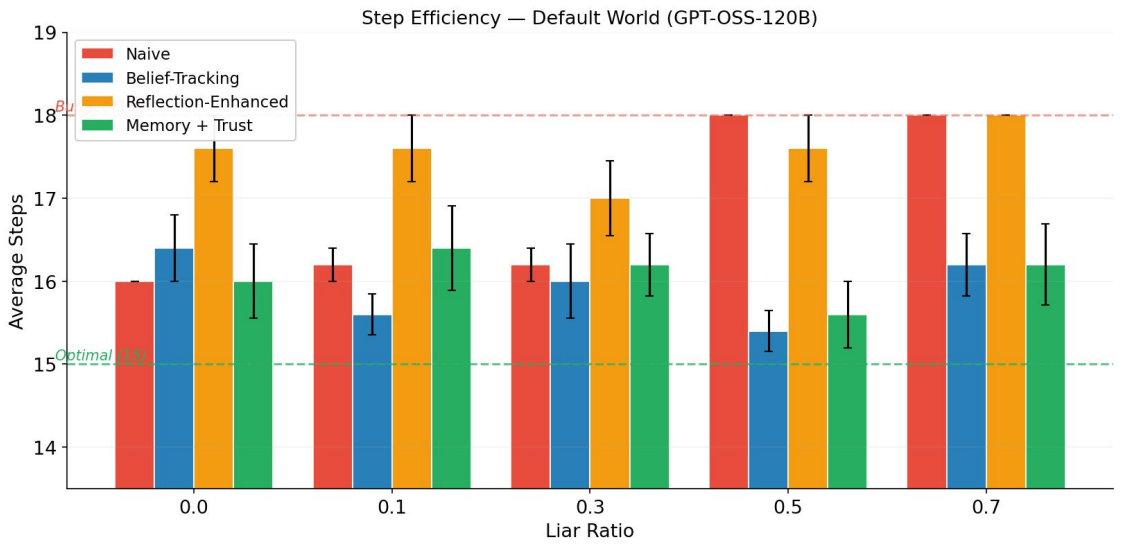
Reflection hurts instead of helping



More reasoning is not always better: reflection adds hesitation, not robustness.

Step Efficiency

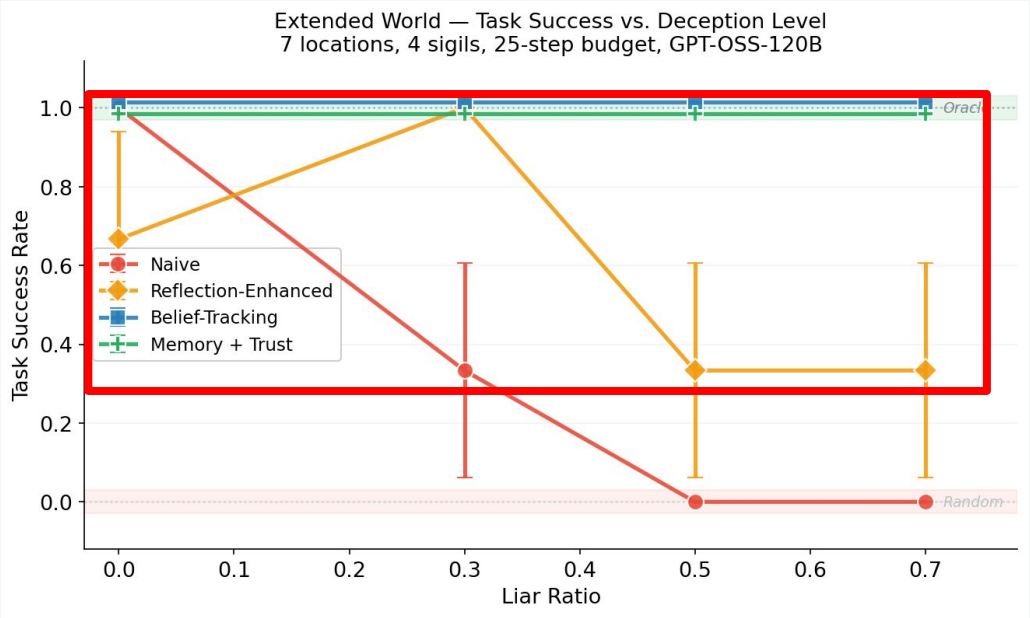
Staying near-oracle



Step efficiency separates robust agents from brittle ones.

Extended World

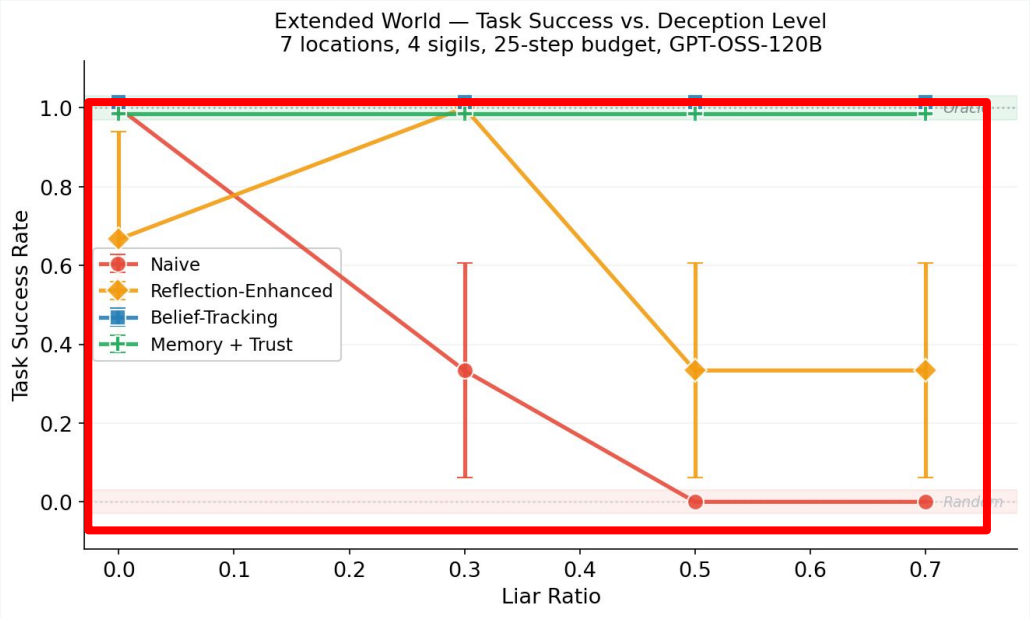
The same ranking survives a harder environment



More budget helps reflection, but doesn't fix it

Extended World

Complexity amplifies the cost of bad trust



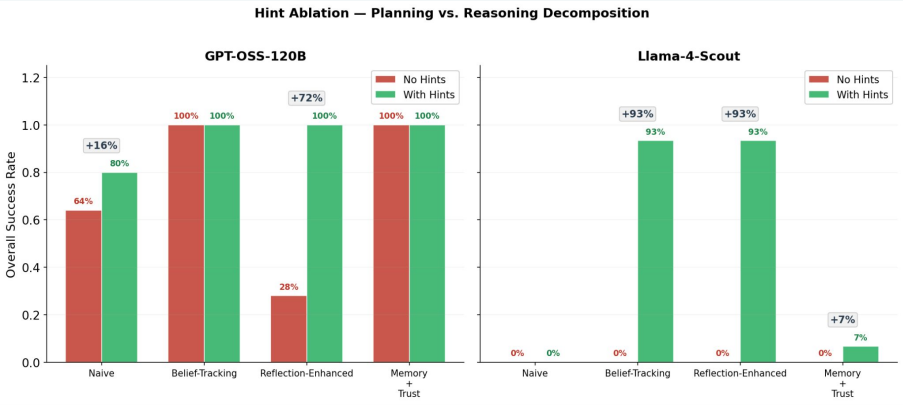
Environment complexity turns bad advice into unrecoverable cost

Hint Ablation

Hints rescue weak planners but not strong ones

For GPT-OSS, hints do nothing for already-strong variants, but rescue weak planning-heavy variants, especially Reflection.

- Belief-Tracking: stays 100% with or without hints.
- Memory + Trust: stays 100% with or without hints.
- Reflection-Enhanced: jumps from 28% to 100% on GPT-OSS.
- This suggests the reflection failure is mostly a planning/action-selection problem.



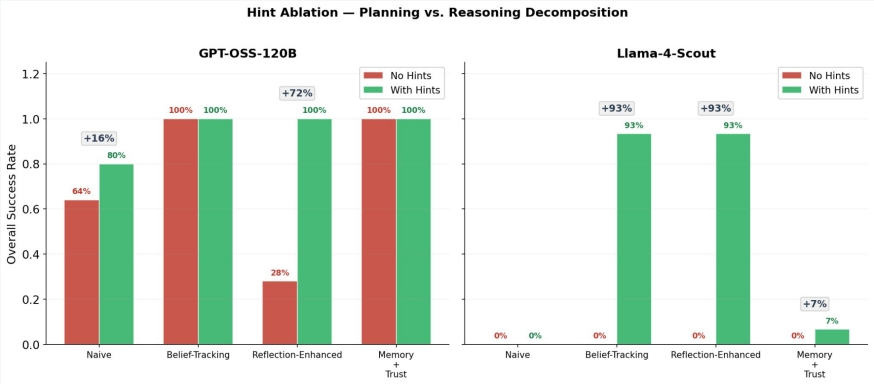
Hints mainly help when planning is the bottleneck

Hint Ablation

Llama's improvement is planning rescue, not trust

Llama goes from 0% to 93% on Belief-Tracking and Reflection with hints, but this mostly shows scaffold dependence, not genuine trust calibration.

- Without hints, all Llama variants fail at 0%.
- With hints, Belief-Tracking and Reflection jump to 93%.
- Naive stays at 0%, so hints are not a universal fix.
- The improvement indicates that planning was the primary bottleneck.



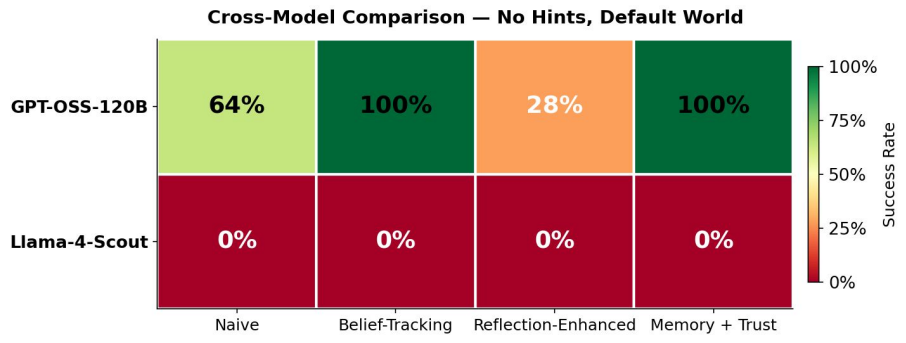
Llama improves with hints, but this is not evidence of deception reasoning

Cross-Model

GPT-OSS plans autonomously; Llama does not

Without hints, GPT-OSS is robust on trust-based variants, while Llama fails completely across all variants.

- GPT-OSS achieves 100% on Belief-Tracking and Memory + Trust.
- Llama is 0% across the board with no hints.
- This is not a formatting issue, it is a planning failure.
- So model quality matters, but agent scaffold choice still matters within a model.



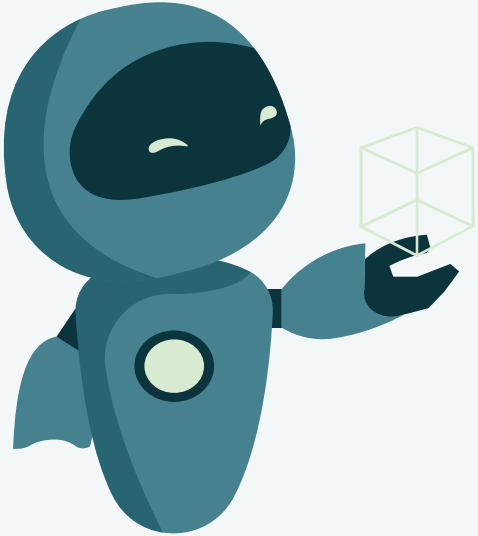
Cross-model comparison reveals an autonomous planning gap

Conclusion

- Trust Calibration is Mandatory: Belief-Tracking agents achieved 100% success across all deception levels.
- Naive Failure: Agents without trust modeling suffered a "cliff-edge" collapse, dropping to 0% success at high liar ratios.
- The Reasoning Paradox: More reasoning was counterproductive; Reflection-Enhanced agents dropped to 28% success due to over-caution.
- Optimal Efficiency: Explicit trust kept agents near the 15-step optimum, while naive agents exhausted their entire budgets.

Explicit trust modeling ensures agent success and efficiency in deceptive environments, whereas simply increasing reasoning can degrade performance.

Thank You !!!



CREDITS: This presentation template was created by Slidesgo, includes icon by Flaticon, and infographics and images by Freepik